

Nexus Hackathon 2026

Affiliated to GLS University, Ahmedabad

PROJECT REPORT

ON

NexusOCR: Multilingual Document OCR Extraction Pipeline

Adaptive Multilingual Document Intelligence Engine

Submitted as part of Nexus Hackathon 2026

Particular	Details
University	GLS University, Ahmedabad
Project Type	Software Project
Project Title	Multilingual Document OCR Extraction Pipeline
Team ID	56
Team Members	Harsh Aalwani [202500719010001] Jayraj Prajapati [202500719010044]
Stage	Nexus Hackathon 2026

ABSTRACT

NexusOCR is an enterprise-oriented document extraction and intelligence system designed to extract text and structure from multilingual business documents without relying on third-party cloud OCR APIs. The project addresses the practical challenge of processing both born-digital documents and scanned documents efficiently, while maintaining support for English and selected Indic scripts including Hindi/Devanagari, Gujarati, and Marathi.

The central design is a dual-tier routing pipeline. A Page Trust Profiler evaluates the page content and routes pages with a high native-text trust score to a fast PyMuPDF extraction path, while scanned or flattened pages are escalated to a PaddleOCR neural vision path. The system further supports table reconstruction, spatial bounding boxes, key-value and common entity extraction, multi-format ingestion, batch processing, REST/CLI access, and an interactive browser interface for visual verification.

The supplied project documentation reports approximately 30 ms per page for the digital fast path and defines performance targets for neural OCR on CPU and GPU. The test suite contains 66 automated tests, of which 65 pass and one optional GPU test is skipped. The architecture is organized into interfaces, pipeline runtime, domain features, reusable processors, engine adapters, and shared contracts, with explicit separation between execution mechanics and domain decisions.

Keywords: OCR, Multilingual Document Processing, Indic Scripts, PyMuPDF, PaddleOCR, FastAPI, Document Layout, Table Extraction, Local-First Processing, Document Intelligence

TABLE OF CONTENTS

Sr. No.	Topics	Section
1	Introduction	1.0
	1.1 College Profile	1.1
	1.2 Project Profile	1.2
	1.2.1 Overview	1.2.1
	1.2.2 User Roles and Descriptions	1.2.2
2	Proposed System	2.0
	2.1 Scope	2.1
	2.2 Objective	2.2
	2.3 Constraints	2.3
	2.3.1 Hardware Constraints	2.3.1
	2.3.2 Software Constraints	2.3.2
	2.4 Advantages	2.4
	2.5 Limitations	2.5
	2.6 Operating Principle	2.6
3	Environment Specification	3.0
	3.1 Hardware & Software Requirements	3.1
	3.2 Development Description	3.2
	3.3 Development and Execution Tools	3.3
	3.4 Deployment Characteristics	3.4
4	System Planning	4.0
	4.1 Feasibility Study	4.1
	4.2 Software Engineering Model	4.2
	4.3 Risk Analysis	4.3
	4.4 Project Schedule	4.4
5	System Analysis	5.0
	5.1 Detailed SRS	5.1
	5.2 System Workflow Analysis	5.2

Sr. No.	Topics	Section
	5.3 Data Flow Diagrams (DFD)	5.3
	5.4 Assumptions	5.4
6	Software Design	6.0
	6.1 Architectural Design	6.1
	6.2 Interface Design	6.2
	6.3 API and CLI Design Summary	6.3
7	Testing	7.0
	7.1 Unit Testing	7.1
	7.2 Integration Testing	7.2
	7.3 Test Result Summary	7.3
	7.4 Overall Verification Status	7.4
8	Future Enhancements	8.0
9	References	9.0
	9.1 Project Documentation Sources	9.1
A	Appendix A - Supported Formats	A
B	Appendix B - Core Project Components	B
C	Appendix C - Report Scope Note	C

1. INTRODUCTION

1.1 Project Profile

Attribute	Details
Project Title	NexusOCR: Multilingual Document OCR Extraction Pipeline
Frontend	Vanilla ES6+, HTML5, responsive CSS3 Grid & Flexbox, SVG Canvas
Backend	Python 3.10+, FastAPI, Uvicorn ASGI
Deep Learning & OCR	PaddleOCR (PP-OCRv4/v6), PyMuPDF (fitz), OpenCV, IBM Docling
Browser Support	Google Chrome, Microsoft Edge, Brave, Mozilla Firefox
Development Platform	VS Code, Zed, Git, PowerShell
Documentation	Markdown, Microsoft Word 2021
Target Hardware	AMD Ryzen/EPYC with AVX vector instructions and NVIDIA CUDA GPU
Deployment Mode	Local-first and offline-capable; no external cloud OCR dependency

1.1.1 Overview

NexusOCR is an enterprise-oriented document extraction and intelligence system intended for high-throughput, local document processing. Its primary architectural contribution is an adaptive dual-tier routing pipeline that avoids using heavyweight visual OCR when reliable native document text is already available.

For born-digital PDF pages, PyMuPDF is used to inspect and extract native text and vector information. For scanned or flattened pages, the system routes work to PaddleOCR for visual text detection and recognition. The documented language scope includes English, Hindi/Devanagari, Gujarati, and Marathi, with spatial coordinates, structural table reconstruction, and key-value extraction included in the processing flow.

1.2 Problem Statement

Document-processing workflows often contain a mix of born-digital PDFs, scanned pages, office documents, images, and structured tables. A single OCR path may waste compute on pages that already contain machine-readable text, while script-limited solutions can struggle with Indic languages and complex layouts. The project therefore targets an adaptive engine that can select an appropriate extraction strategy per page while keeping document data on the host machine.

1.3 Need for the System

- Reduce unnecessary OCR computation on pages where native text can be extracted directly.
- Provide broader multilingual coverage for regional Indic scripts alongside English.
- Preserve spatial and tabular structure instead of returning only an unstructured text stream.
- Support local processing for workflows where sending sensitive documents to external services is undesirable.
- Expose the same core processing capabilities through a browser UI, REST API, and CLI.

2. PROPOSED SYSTEM

2.1 Scope

NexusOCR provides an end-to-end document parsing service that combines multi-format ingestion, page-level adaptive routing, Indic language recognition, structural table handling, entity extraction, and exportable structured results. The system is designed as a reusable pipeline component rather than a complete business workflow application.

2.1.1 Functional Scope

- Adaptive dual-tier processing of native and scanned pages.
- Recognition of English, Hindi/Devanagari, Gujarati, and Marathi scripts.
- Ingestion of PDF, DOCX, XLSX, CSV, PPTX, TIFF, PNG, JPG, BMP, WebP, TXT, HTML, and EPUB files.
- Structural table reconstruction into Markdown and HTML-compatible representations.
- Extraction of key-value pairs, dates, currency values, email addresses, and phone numbers.
- Defensive rejection of unsupported audio and video payloads.
- Dual-pane browser inspection with SVG bounding boxes, synchronized pan/zoom, and data export.
- Batch processing with fault isolation and REST/CLI access.

2.2 Objective

The primary objective is to eliminate dependence on third-party cloud OCR APIs for the targeted document-processing workload by providing an offline, high-speed, and multilingual document intelligence engine.

- Sub-80 ms fast path for suitable born-digital pages.
- At least 98% recognition accuracy as a stated target for printed Indic languages and dense tabular layouts.
- 100% local processing during document analysis.
- Hardware flexibility across CPU AVX execution and NVIDIA CUDA acceleration.
- A lightweight browser UI based on native web technologies.

2.3 Constraints

2.3.1 Hardware Constraints

Specification	Minimum	Recommended
Operating System	Windows 10 64-bit / Linux Ubuntu 20.04+	Windows 11 64-bit / Linux Ubuntu 22.04+
Processor	Intel Core i3 10th Gen or AMD Ryzen 5	AMD Ryzen 5/7 or Intel Core i7 with AVX2/AVX-512
RAM	4 GB	8-16 GB DDR4/DDR5
Storage	256 GB SSD with at least 2 GB free	512 GB NVMe SSD
GPU	Not required	NVIDIA GeForce RTX 3060/4060, 8 GB VRAM

2.3.2 Software Constraints

- Python runtime: 3.10, 3.11, or 3.12 (64-bit); the architecture documentation also emphasizes deterministic Python 3.10.11 compatibility.
- Supported browsers include current Chrome, Edge, Brave, and Firefox versions consistent with the project test environment.
- No network connection is required during document analysis because the project is designed for local execution.

2.4 Advantages

1. Fast digital extraction: the documented Tier 1 path processes suitable PDF pages in approximately 30-60 ms.
2. Indic language coverage for the targeted Hindi/Devanagari, Gujarati, Marathi, English, etc workloads.
3. Local-first processing that keeps document analysis on the host machine.
4. Hardware-aware execution across CPU and optional CUDA acceleration.
5. One processing entry point for common office, image, markup, and PDF formats.
6. Structured outputs that preserve tables and normalized spatial coordinates.

2.5 Limitations

1. Severely low-DPI or heavily degraded scans may produce lower character confidence.
2. Current recognition is optimized for printed and semi-structured typography; unconstrained cursive handwriting is listed as a future escalation target.
3. Initial loading of neural weights on CPU introduces approximately 1.5-2 seconds of startup latency.
4. Large, uncompressed multi-page TIFF archives can consume substantial RAM.

2.6 Operating Principle

The system first validates and identifies the input format. For PDF content, pages are inspected and assigned a trust score. High-trust pages are processed by the native vector path; lower-trust pages are routed to neural OCR. Results from both paths are normalized into common document contracts, enriched with entities and layout information, and synthesized into a single DocumentResult representation that the UI, API, and CLI can consume.

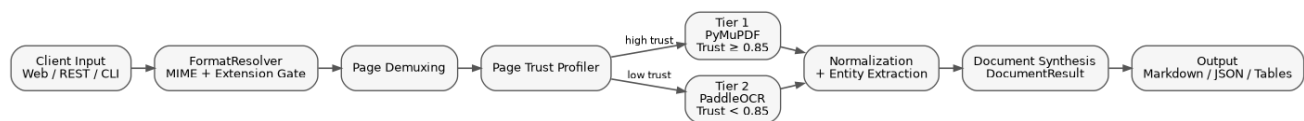


Figure 2.6.1 - NexusOCR adaptive document processing workflow

3. ENVIRONMENT SPECIFICATION

3.1 Hardware & Software Requirements

3.1.1 Development Environment

Item	Configuration
Processor	AMD Ryzen 5 / Intel Core i7 (AVX supported)
System RAM	8 GB minimum; 16 GB recommended
Storage	512 GB NVMe SSD, 1 TB HDD
Acceleration	NVIDIA GeForce RTX 4060, 8 GB GDDR6 VRAM (development benchmark environment)
Operating System	Windows 11 Pro 64-bit

3.1.2 Client Configuration

Item	Configuration
Web Browsers	Chrome 134+, Edge 134+, Brave 1.76+, Firefox
Display	1280 x 720 minimum; 1920 x 1080 recommended
Input Devices	Mouse with scroll wheel / trackpad

3.2 Development Description

Component	Technology	Version	Purpose
Backend Framework	FastAPI	0.110+	REST API routing, OpenAPI generation, request lifecycle management
ASGI Server	Uvicorn	0.28+	High-performance asynchronous server runtime
Data Validation	Pydantic	2.0+	Typed validation and JSON schema serialization
Digital PDF Parser	PyMuPDF (fitz)	1.24+	Native text extraction and PDF rasterization
Neural Vision OCR	PaddleOCR	3.7+	Text detection and Indic text recognition
Deep Learning Engine	PaddlePaddle	3.0+	Tensor execution with CPU/GPU backends
Document Layout	IBM Docling	2.126+	Reading order and table extraction
Computer Vision	OpenCV	4.8+	Preprocessing, contrast analysis, DPI scaling

Component	Technology	Version	Purpose
Office Parsers	python-docx / openpyxl / python-pptx	1.0+	OpenXML document and spreadsheet parsing
Frontend	Vanilla ES6+	Native	Responsive UI, SVG overlays, pan/zoom controls

3.3 Development and Execution Tools

- Visual Studio Code for development and debugging.
- Git for version control and branch-based development.
- PowerShell for Windows development workflows.
- pytest for automated unit and integration verification.
- FastAPI-generated OpenAPI documentation for API discovery and manual verification.

3.4 Deployment Characteristics

The supplied project documentation defines a local-first deployment model. The engine is intended to run on the host hardware with no required cloud OCR dependency. CPU execution is supported, while NVIDIA CUDA can be used as an acceleration path when available.

4. SYSTEM PLANNING

4.1 Feasibility Study

A. Technical Feasibility

NexusOCR uses mature open-source components and a modular architecture. PyMuPDF provides a native PDF extraction path, while PaddleOCR supplies neural document recognition for scanned pages. FastAPI and Uvicorn expose the processing service, and the frontend uses native browser technologies with SVG overlays. The architecture is therefore technically feasible for the stated offline and performance-oriented goals.

B. Operational Feasibility

The system exposes both a graphical browser interface and programmatic entry points. Document analysts can upload and inspect files visually, while integration services can use REST endpoints or the CLI. The workflow is designed to minimize the amount of training needed for routine document processing.

C. Economic Feasibility

A local-first approach avoids recurring per-page OCR charges from external cloud providers. The implementation uses open-source libraries, which supports a lower software operating cost for the target deployment model. Hardware remains the main infrastructure consideration.

D. Legal & Privacy Feasibility

The project is designed around local document processing and therefore avoids transmitting analysis payloads to external OCR services. This supports privacy-sensitive deployments. The report uses the project documentation's framing of compliance-oriented local processing; specific legal compliance should still be validated separately for each deployment context.

E. Schedule Feasibility

The project was organized into iterative milestones covering requirements, architecture, engine integration, multi-format processing, UI development, and test verification. The supplied schedule indicates completion across a nine-week development timeline.

4.2 Software Engineering Model

NexusOCR follows a Prototyping & Iterative Enhancement Model. The approach is suitable because OCR quality, routing thresholds, format coverage, and UI inspection behavior benefit from repeated evaluation against representative document samples.

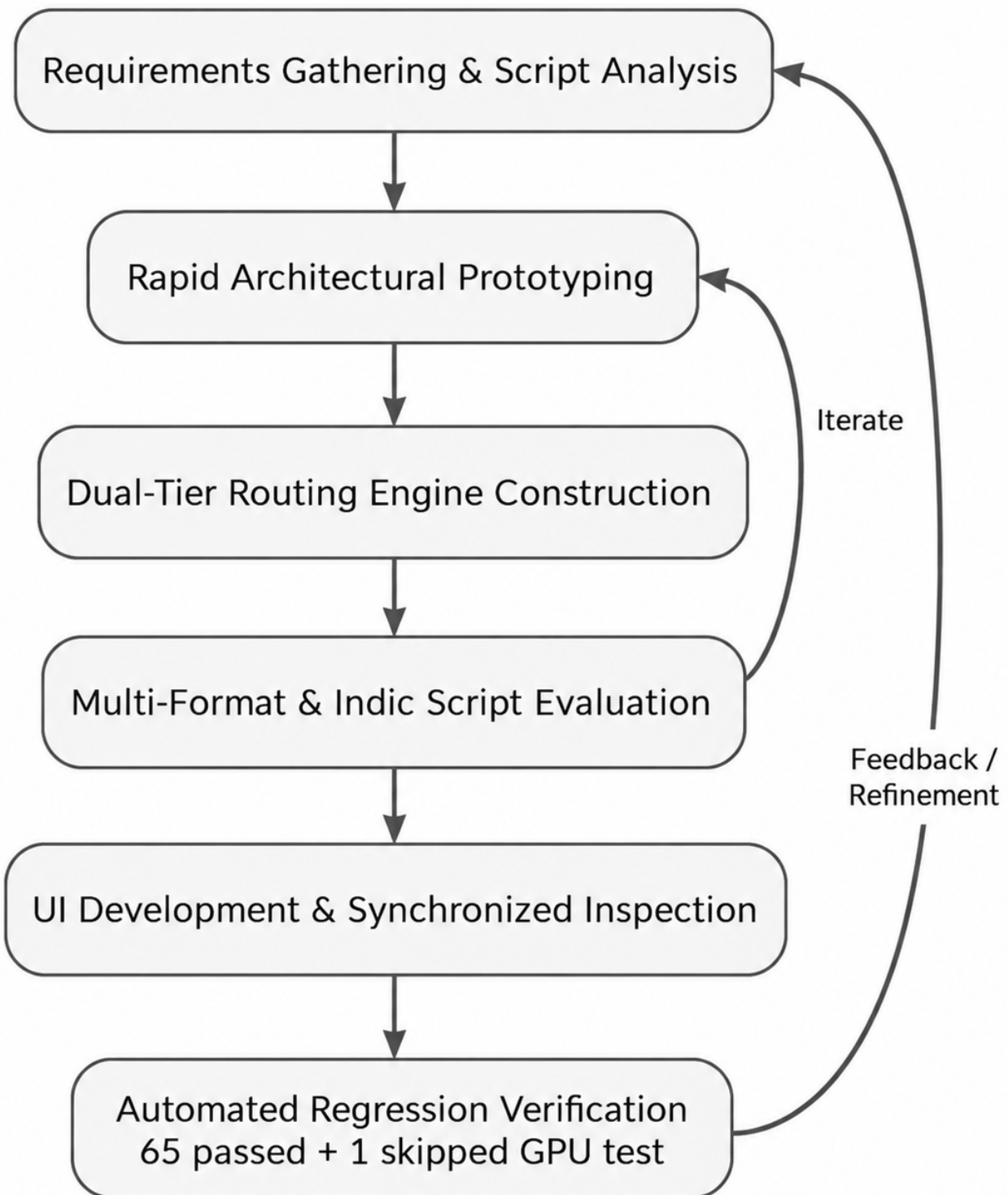


Figure 4.2.1 - Prototyping & Iterative Enhancement lifecycle

4.2.1 Engineering Phases

1. Requirements Gathering & Analysis: identify document formats, Indic language requirements, and performance targets.
2. Rapid Prototyping: Candidate OCR and document-processing approaches such as Ollama VLM, RapidOCR, Docling, and PaddleOCR were evaluated for latency, resource usage, and suitability. The final core pipeline uses PyMuPDF and PaddleOCR.
3. Core Engine Implementation: implement DocumentOCRService, Page Trust Profiler, and dual-tier routing.
4. Refinement & Testing: integrate OpenXML processors, defensive MIME validation, and CPU execution improvements.
5. Final Integration: complete the dual-pane UI and the automated regression suite.

4.2.2 Architecture Principles

- Strict separation of execution mechanics from domain decisions: pipeline/ controls execution while features/ controls domain behavior.
- Hardware and provider isolation: heavyweight dependencies remain in engines/ and are accessed through abstractions and typed contracts.
- Non-intrusive backward compatibility: legacy import paths are preserved through lightweight bridges.
- Deterministic Python3.10.11 compatibility is guarded through AST-based checks.

4.3 Risk Analysis

Risk	Impact	Mitigation
Indic script ligature defects	High	Use representative Gujarati/Hindi fixtures and early unit testing.
High CPU latency without GPU	High	Use page trust profiling to reserve neural OCR for scanned/flattened pages.
Unsupported multimedia uploads	Medium	Reject audio/video payloads at the FormatResolver MIME gate.
File upload and path exploits	Medium	Sanitize extensions, process streams safely, and verify magic bytes.

4.4 Project Schedule

4.4.1 Task Dependency

Task	Depends On	Responsible Component
MIME Validation & Ingestion	Incoming HTTP / CLI request	FormatResolver
Page Trust Profiling	Ingestion and page demuxing	pymupdf.py
Tier 1 Vector Extraction	Trust score ≥ 0.85	extract_digital_page()
Tier 2 Neural Vision OCR	Trust score < 0.85	PaddleOCREngine
Key-Value Form Extraction	Raw text normalization	FormDataExtractor

Task	Depends On	Responsible Component
Script Tagging & Language	Unicode text stream	DefaultTranslationEngine
Document Synthesis	All pages processed	DocumentResult schema
UI Side-by-Side Rendering	Successful API response	Vanilla ES6+ web client

4.4.2 Timeline Chart

Task / Milestone	W1	W2	W3	W4	W5	W6	W7	W8	W9
Requirements & Literature Study	■	■							
Architecture Design & Trust Gates		■	■						
PaddleOCR Indic & PyMuPDF Engine			■	■	■				
Office & Multi-Format Processors					■	■			
REST API & CLI Implementation						■	■		
UI Design & Interactive SVG Canvas							■	■	
Automated Testing (66 Tests) & Docs								■	■

5. SYSTEM ANALYSIS

5.1 Detailed Software Requirements Specification (SRS)

5.1.1 Functional Requirements

ID	Requirement	Specification
FR-01	Universal Document Ingestion	Ingest PDF, DOCX, XLSX, PPTX, CSV, PNG, JPG, TIFF, BMP, WebP, TXT, HTML, and EPUB files.
FR-02	Defensive Media Filtering	Inspect file signatures and reject unsupported audio/video payloads with HTTP 400 responses.
FR-03	Automated Page Trust Profiling	Compute a page-level trust score using character density and printable-glyph ratios.
FR-04	Dual-Tier Execution Routing	Route pages with trust ≥ 0.85 to Tier 1 and pages below 0.85 to Tier 2.
FR-05	Multilingual Indic Recognition	Detect and transcribe Hindi/Devanagari, Gujarati, Marathi, and English with confidence information.
FR-06	Spatial Bounding Box Extraction	Represent extracted text using normalized xmin, ymin, xmax, ymax coordinates.
FR-07	Table Grid Preservation	Reconstruct tabular structures for Markdown and HTML-style output.
FR-08	Form Entity Extraction	Extract key-value pairs, dates, currencies, email addresses, and phone numbers.
FR-09	Synchronized Web UI	Provide side-by-side document rendering, interactive SVG boxes, and extracted Markdown.

5.1.2 Non-Functional Requirements

ID	Category	Specification
NFR-01	Performance	Tier 1 target under 30 ms per page (reported ~11 ms); Tier 2 target under 1.5 s on host CPU and under 400 ms on GPU.
NFR-02	Security & Privacy	All document analysis runs locally with zero outbound network requests required by the processing path.
NFR-03	Reliability	Batch failures are isolated so one failed document does not stop the remaining batch.
NFR-04	Usability	The UI runs with native HTML5/ES6+ technologies without browser plugins.
NFR-05	Maintainability	Automated regression coverage includes unit, integration, and CLI workflows.

5.1.3 Input and Output Specification

Area	Input	Output
Document Input	Supported files and validated upload streams	Structured processing request
Page Analysis	PDF page content, raster pages, extracted glyphs	Trust score + selected extraction tier
OCR Extraction	Detected text regions and images	Text lines, confidence, bounding boxes
Synthesis	Normalized page results	DocumentResult JSON and Markdown
UI Export	Selected processed result	Copied/downloaded Markdown or JSON

5.2 System Workflow Analysis

The system workflow can be viewed as a controlled sequence of validation, profiling, routing, extraction, normalization, synthesis, and export. The routing decision occurs at page level, allowing mixed documents to use different extraction methods within a single processing job.

5.3 Data Flow Diagrams (DFD)

5.3.1 Level 0: Context Data Flow Diagram

The Level 0 DFD represents NexusOCR as one processing boundary. External clients provide document inputs and receive standardized extraction results.



Figure 5.3.1 - Level 0 Context Data Flow Diagram

5.3.2 Level 1: Subsystem Data Flow Diagram

The Level 1 diagram decomposes the processing boundary into ingestion, trust profiling, two extraction tiers, and semantic synthesis. The two tier-specific branches merge again at the synthesis stage so that all pages share a common output contract.

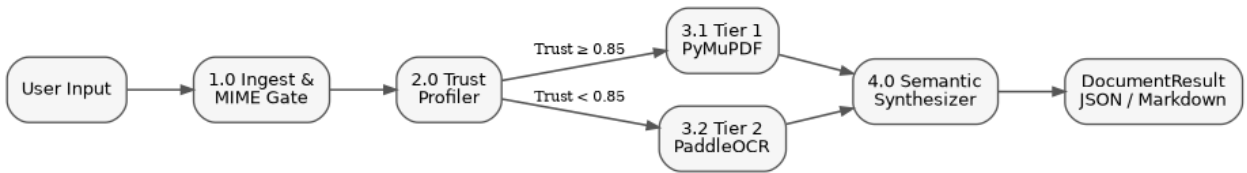


Figure 5.3.2 - Level 1 Subsystem Data Flow Diagram

5.3.3 Level 2: Detailed Processing Data Flow Diagram

The Level 2 view focuses on the neural extraction path: page rasterization and de-skewing, DBNet detection, Indic sequence transcription, entity matching, SVG coordinate normalization, and structured page result generation.

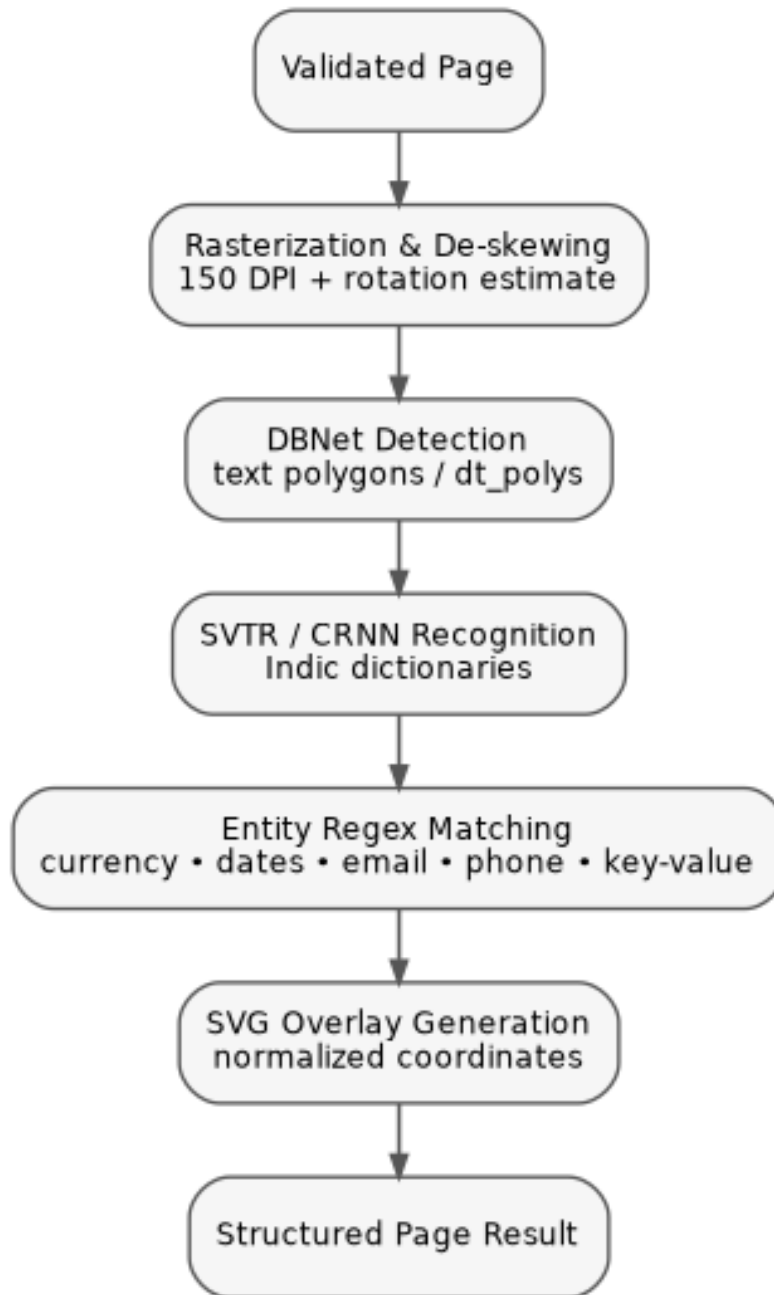


Figure 5.3.3 - Level 2 Detailed Processing and Entity Synthesis

5.4 Assumptions

- Input documents are expected to be within the supported format set and accessible to the local processing environment.
- The accuracy target applies to representative printed document workloads and is not a guarantee for every document condition.
- GPU acceleration is optional; the CPU path remains a supported operating mode.
- Complex handwriting is outside the current core recognition scope and is treated as a future enhancement area.

6. SOFTWARE DESIGN

6.1 Architectural Design

The README architecture describes NexusOCR as a Hybrid Feature-Oriented Pipeline. The design separates client-facing interfaces, pipeline execution mechanics, domain features, reusable document processors, engine adapters, and shared contracts. This separation limits the impact of replacing an OCR engine or adding a new processing stage.

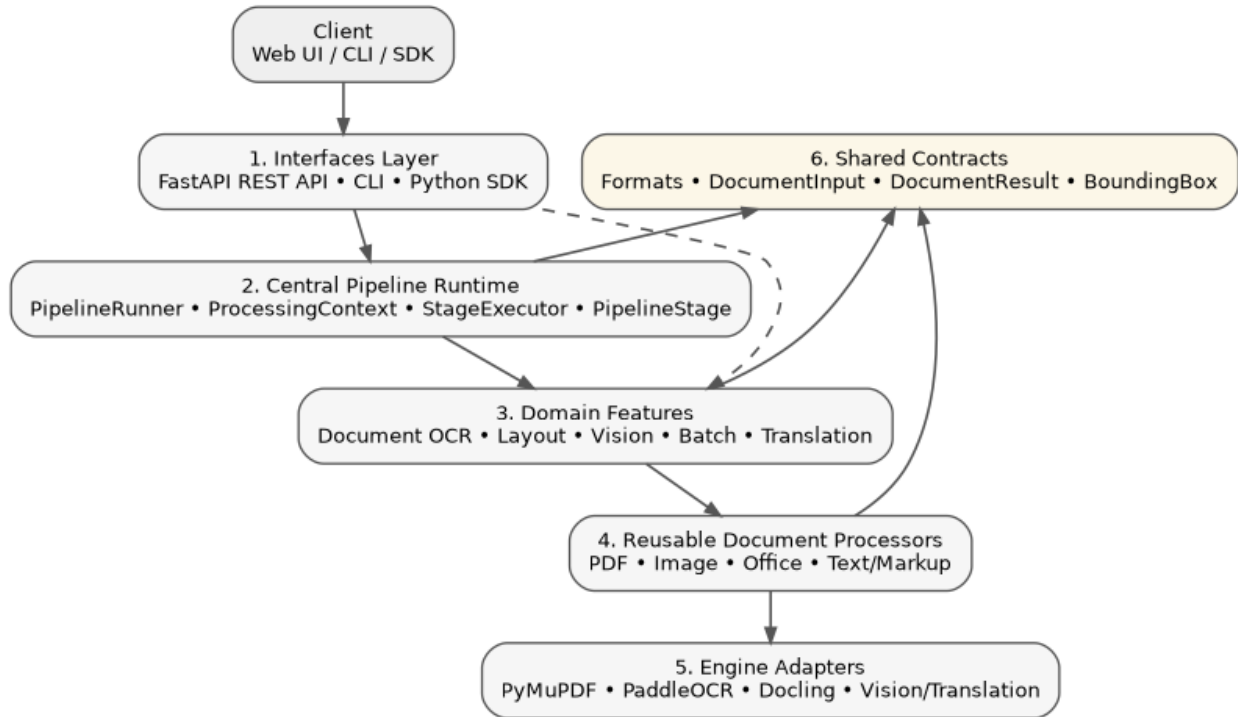


Figure 6.1.1 - NexusOCR architectural layer model based on the supplied engineering documentation

6.1.1 Interfaces Layer

- FastAPI REST endpoints provide HTTP-based processing and health interfaces.
- CLI commands expose process, formats, health, and serve workflows.
- Python SDK integration provides programmatic access from Python clients.

6.1.2 Central Pipeline Runtime

- PipelineRunner coordinates stage execution.
- ProcessingContext carries request-scoped execution data.
- StageExecutor manages stage execution mechanics such as timing, cancellation, progress, and errors.
- PipelineStage defines the common abstraction for pipeline stages.

6.1.3 Domain Features

- document_ocr/ handles multi-format OCR and form extraction.
- layout/ provides layout and table-related processing through Docling/TableFormer components.
- vision/ supports document inspection and preview workflows.
- batch.py provides isolated multi-file queue handling.

- translation/ provides script detection and translation-related abstractions.

6.1.4 Reusable Processors

Processor	Responsibility
pdf.py	PyMuPDF-based PDF input/output and page handling.
image.py	Pillow-based image handling, including multi-frame TIFF processing.
office.py	Processing for Word, Excel, and PowerPoint documents.
text_markup.py	Handling text, HTML, Markdown, and EPUB formats.

6.1.5 Engine Adapters

- PyMuPDF adapter for native PDF extraction.
- PaddleOCR adapter for neural visual OCR and optional CUDA execution.
- Docling adapter for document layout and table processing.
- Abstract vision and translation adapter interfaces for provider isolation.

6.1.6 Shared Contracts

- FormatResolver and capabilities definitions for supported input types and validation.
- DocumentInput and ProcessingOptions for strongly defined requests.
- DocumentResult, PageResult, and BoundingBox for standardized outputs.

6.1.7 Backward Compatibility

The architecture retains legacy import paths through lightweight compatibility shims. This reduces disruption while moving the implementation toward the modular pipeline structure described in the README engineering documentation.

6.2 Interface Design

The user interface is implemented with native ES6+, HTML5, responsive CSS3, and SVG. It is organized around an Overview area for system visibility and a Studio Workspace for document inspection. The interface emphasizes direct visual verification between source pages and extracted structure.

6.2.1 Overview Dashboard & System Architecture

The overview screen presents hardware telemetry such as CPU AVX detection and CUDA readiness, the architectural flow, supported formats, and latency counters.

6.2.2 Studio Workspace & Test Fixture Suite

The studio provides a drag-and-drop upload area and pre-loaded fixtures for multilingual OCR, financial tables, and office-document verification.

6.2.3 Dual-Pane Synchronized Document Inspector

The inspector renders the original page and extracted Markdown together. This supports immediate comparison between source layout and reconstructed content.

6.2.4 Interactive SVG Bounding Box Canvas

Text regions can be inspected through bounding-box overlays. Hover and selection behavior connects a spatial region on the page to the corresponding extracted text or table row.

6.2.5 Financial Table & Grid Extraction View

Dense multi-column financial layouts are presented as reconstructed tables so that column relationships remain visible in the output representation.

6.2.6 Multilingual Indic Script Extraction

The interface demonstrates Hindi/Devanagari and Gujarati OCR results, including conjunct and diacritic reconstruction.

6.2.7 Key-Value Form & Entity Inspection

The entity panel displays extracted currency amounts, dates, email addresses, phone numbers, and key-value fields alongside document metadata.

6.2.8 Data Export

The export workflow provides copy and download actions for Markdown and JSON payloads, including structured bounding boxes and processing metadata.

6.3 API and CLI Design Summary

Interface	Representative Operations	Purpose
REST API	POST /api/process, POST /api/batch	Single and batch document processing
REST API	GET /api/page_image	Render page images for inspection
REST API	GET /api/samples	Access test/sample fixtures
REST API	GET /api/health	Health and readiness status
CLI	nexusocr process	Process document from command line
CLI	nexusocr formats	List supported formats
CLI	nexusocr health	Check service health
CLI	nexusocr serve	Start the service

7. TESTING

Testing is the systematic validation of the system against its requirements and expected behavior. NexusOCR uses pytest-based automated verification across individual modules, integrated pipelines, REST endpoints, and CLI workflows.

Metric	Result
Total automated tests	66
Tests passed	65
Tests skipped	1 (GPU optional)
Reported success rate on executed tests	100%
Execution time	~70 seconds

7.1 Unit Testing

Unit testing evaluates isolated functions, classes, and modules. The project documentation identifies the following core test areas:

- FormatResolver: magic-byte signatures, extension mapping, and format categorization across supported file types.
- Page Trust Profiler: threshold behavior for digital and scanned pages and protection against blank-page edge cases.
- PyMuPDF extractor: vector text block extraction, coordinate scaling, and heading classification.
- PaddleOCR adapter: text detection polygons, line rotation correction, and Indic character decoding.
- FormDataExtractor: currency, date, email, phone, and key-value regular-expression patterns.
- DefaultTranslationEngine: Unicode range categorization for Hindi, Gujarati, and Latin scripts.

7.1.1 Defects Fixed During Unit Testing

1. Windows path escaping issues when running under PowerShell.
2. Phantom bounding boxes caused by zero-length whitespace spans.
3. Pydantic v2 schema deprecation issues, including replacement of deprecated `.dict()` usage with `.model_dump()`.
4. Division-by-zero protection for blank document pages in trust scoring.

7.2 Integration Testing

Integration testing verifies interactions among modules, external libraries, API routes, and command-line entry points.

7.2.1 Integration Areas

- Station 01 to Station 04 end-to-end pipeline flow from upload through synthesis.
- Office document parsing using `python-docx`, `openpyxl`, and `python-pptx`.
- FastAPI routes for processing, batching, page imagery, sample access, and health checks.
- Defensive rejection of MP3/WAV and MP4/MKV inputs with immediate HTTP 400 responses.
- CLI subprocess verification for `process`, `formats`, `health`, and `serve` commands.

7.2.2 Integration Defects Fixed

1. Stubbed unneeded ModelScope modules to avoid external import/network startup behavior.
2. Resolved multi-frame TIFF page-order desynchronization.
3. Tuned multipart streaming behavior to support multi-page PDF uploads up to 50 MB without out-of-memory failures.

7.3 Test Result Summary

Test Layer	Focus	Outcome
Unit	Core functions, scoring, extraction adapters, entities	Passed across documented unit areas
Integration	API, multi-format processing, pipeline routing	Passed across documented integration areas
CLI	Command execution and service workflows	Passed
GPU-specific optional test	CUDA-dependent behavior	Skipped when optional GPU path is unavailable

7.4 Overall Verification Status

The documented final suite contains 66 automated tests with 65 passed and one skipped optional GPU test. This indicates that the core local/CPU workflow and the documented integration paths were verified by the supplied test suite, while GPU-specific behavior remains conditional on the test environment.

8. FUTURE ENHANCEMENTS

8.1 PostgreSQL and pgvector Semantic Document Archive

Integrate persistent PostgreSQL storage with pgvector to retain document embeddings and enable semantic search and retrieval across larger document repositories. This would extend the engine from extraction toward searchable document intelligence.

8.2 Handwritten Indic VLM Escalation Tier

Introduce a lightweight local Vision-Language Model as an escalation tier for complex, unconstrained regional handwriting such as prescriptions and examination papers. This is a future enhancement only; it is not part of the current core detection path.

8.3 Real-Time WebSocket Token and Page Streaming

Stream extracted page blocks and tokens incrementally to clients as processing progresses, improving responsiveness for large multi-page documents.

8.4 Containerized Deployment

Provide Docker or Podman images with appropriate CPU libraries and optional CUDA runtimes for repeatable deployment in enterprise, cloud, and Kubernetes environments.

8.5 Advanced Document Layout Models

Extend table reconstruction to nested or borderless financial layouts using newer layout and table-model capabilities.

8.6 Automated Sensitive-PII Redaction

Add automated redaction before export for sensitive identifiers such as Aadhaar numbers, PAN information, and credit-card numbers.

9. REFERENCES

Reference	URL
FastAPI Documentation	https://fastapi.tiangolo.com/
PyMuPDF Documentation	https://pymupdf.readthedocs.io/
PaddleOCR Repository and Documentation	https://github.com/PaddlePaddle/PaddleOCR
PaddlePaddle Official Packages	https://www.paddlepaddle.org.cn/
IBM Docling Documentation	https://github.com/DS4SD/docling
Pydantic Documentation	https://docs.pydantic.dev/latest/
OpenCV Documentation	https://docs.opencv.org/
Uvicorn ASGI Web Server	https://www.uvicorn.org/
python-docx Documentation	https://python-docx.readthedocs.io/
openpyxl Documentation	https://openpyxl.readthedocs.io/
python-pptx Documentation	https://python-pptx.readthedocs.io/
pytest Documentation	https://docs.pytest.org/

9.1 Project Documentation Sources

- NexusOCR PROJECT_REPORT.md - supplied project report source used as the primary content basis for this report.
- NexusOCR README - supplied architecture and engineering documentation used to expand the software design section.
- Project Report PDF reference - supplied as a formatting and report-structure reference for the academic presentation style.

APPENDIX A - SUPPORTED FORMATS

Category	Supported Formats
PDF	PDF
Office	DOCX, XLSX, PPTX
Data	CSV
Images	TIFF, PNG, JPG, BMP, WebP
Text / Markup	TXT, HTML, EPUB
Rejected Media	MP3, WAV, MP4, MKV

APPENDIX B - CORE PROJECT COMPONENTS

Layer	Representative Components
Interfaces	FastAPI REST, CLI, Python SDK
Pipeline	PipelineRunner, ProcessingContext, StageExecutor, PipelineStage
Features	document_ocr, layout, vision, batch, translation
Processors	pdf.py, image.py, office.py, text_markup.py
Engines	pymupdf.py, paddleocr.py, docling.py, vision/base.py, translation/base.py
Contracts	formats.py, results.py, input.py

APPENDIX C - REPORT SCOPE NOTE

This report has been expanded from the supplied NexusOCR project report and README while preserving their terminology, architecture, functional scope, testing results, and future roadmap. UML diagrams and database design are not included as standalone report chapters because the supplied project report explicitly marked those sections as skipped. Current VLM/LLM-based detection is not presented as part of the core system; VLM usage appears only as a future handwritten-text escalation enhancement in the supplied material.